Title: **An End-to-End Neural Information Retrieval and Extractive Question Answering System**

Group Members

Name	***ID***
Mahlet Fiseha	**GSE/2648/15**
Abraham Bekele	**GSE/5265/14**
Abaynesh Dessie	**GSE/6121/15**

Table of Contents

Abstract

This project presents the design and implementation of a complete end-to-end information retrieval and extractive question answering system. The system leverages the MS MARCO dataset, a large-scale collection of real web passages and queries, to build a retrieval-based QA pipeline. The core components include exploratory data analysis (EDA) to understand dataset statistics, a BM25 baseline retriever for initial passage ranking, a KNRM (Kernel-based Neural Ranking Model) re-ranker implemented from scratch to refine retrieval results, and a pre-trained DistilBERT model (fine-tuned on SQuAD 2.0) for extractive answer generation.

The EDA reveals that queries average 6.1 words, passages average 70–72 words, and each query typically has 1–2 relevant passages, providing a solid foundation for retrieval and answer extraction. The BM25 retriever efficiently selects top-k candidate passages using term frequency and inverse document frequency, achieving reasonable ranking quality. The KNRM model, which computes cosine similarity between word embeddings followed by RBF kernel pooling, is trained using pairwise ranking loss to learn relevance scores; however, training was optionally skipped for time constraints, demonstrating flexibility for different resource settings.

For answer extraction, the system uses DistilBERT, a lightweight transformer model (65 million parameters), to extract answer spans from retrieved passages. The pipeline was tested with both interactive and batch question answering modes. On a batch of 10 diverse questions (e.g., “What is deep learning?”, “How does speech recognition work?”), the system achieved an average confidence of 0.711 (71.1%), with high-confidence answers (>0.7) for 6 out of 10 questions. Visualizations of confidence distribution, answer length, and question length vs. confidence are provided to evaluate performance.

The implementation includes a user-friendly interactive interface with quick example buttons, batch processing capabilities, and performance evaluation metrics. The system demonstrates the feasibility of combining classical BM25 retrieval with neural re-ranking and transformer-based extractive QA, achieving reasonable accuracy while maintaining computational efficiency. Future work could focus on training the KNRM model on larger data, integrating dense retrieval (e.g., DPR), and deploying the system as a web service.

Chapter One

Project Overview

1. Introduction

The exponential growth of digital information has created an urgent need for intelligent systems that can efficiently retrieve and synthesize relevant information from vast text corpora. Traditional keyword-based search engines, while effective for document retrieval, often fall short when users seek precise answers to natural language questions. This gap has driven significant research interest in question answering (QA) systems—automated systems that not only locate relevant documents but also extract exact answers from them.

The MS MARCO (Microsoft Machine Reading Comprehension) dataset has emerged as a benchmark for real-world question answering, containing over 8.8 million passages and 1 million real user queries sourced from Bing search logs. Unlike earlier datasets that assume the answer is always present in a single passage, MS MARCO reflects the realistic scenario where answers may be absent, and multiple passages may contain relevant information. This makes it ideal for developing robust retrieval and reading comprehension systems.

This project focuses on building an end-to-end extractive question answering pipeline that combines classical information retrieval techniques with modern deep learning approaches. The system operates in two stages: first, a retrieval stage that identifies top-k candidate passages from a large corpus using BM25, a probabilistic retrieval model; second, a re-ranking and answer extraction stage that uses a neural re-ranker (KNRM) and a pre-trained transformer model (DistilBERT) to extract precise answers.

The choice of BM25 for initial retrieval is motivated by its proven effectiveness, efficiency, and interpretability. It serves as a strong baseline that can be easily replaced or augmented with dense retrieval methods. The KNRM (Kernel-based Neural Ranking Model) re-ranker, implemented from scratch, uses radial basis function kernels to capture different levels of matching between query and document terms, from exact matches to partial semantic matches. Finally, DistilBERT, a distilled version of BERT with significantly fewer parameters (65M vs. 340M), provides fast and accurate answer span extraction while maintaining competitive performance on SQuAD 2.0.

The project also includes comprehensive exploratory data analysis (EDA) of the MS MARCO dataset, revealing key statistics such as average query length (6.1 words), average passage length (70–72 words), and relevance distribution (average 1.11 relevant passages per query). These insights guided hyperparameter choices and evaluation strategies.

An interactive graphical interface was developed using IPython widgets, allowing users to type natural language questions and receive answers with confidence scores. The system also supports batch processing for evaluating multiple questions and saving results to CSV and text reports. Performance was evaluated on a set of 10 diverse technology-related questions, achieving an average confidence of 0.711 (71.1%), with 6 out of 10 answers classified as high confidence (>0.7).

This introduction sets the stage for a detailed discussion of the problem, objectives, and methodology that follow.

2. Statement of the Problem

2.1. The Information Overload Challenge

In the digital age, the volume of text data generated daily—from web pages, scientific articles, social media, and enterprise documents—far exceeds human capacity to process manually. Users seeking specific information often face two interlinked problems: (1) retrieving documents that are relevant to their information need, and (2) locating the exact answer within those documents. Traditional search engines address the first problem by ranking documents based on keyword matching, but they typically return long lists of links, leaving users to read through multiple documents to find answers. This is inefficient for factoid questions such as "What is the capital of France?" or "How does deep learning work?"

2.2. Limitations of Pure Retrieval Systems

Pure retrieval systems (e.g., TF-IDF, BM25) are based on term frequency and document frequency statistics. While they are computationally efficient and interpretable, they suffer from several limitations:

- **Lexical gap**: They cannot capture semantic similarity between different words (e.g., "automobile" vs. "car").
- **No answer extraction**: They rank passages but do not pinpoint the exact answer text.
- **Sensitivity to query formulation**: A poorly phrased query may retrieve irrelevant documents even if relevant content exists.
- **Length normalization issues**: Short passages may be unfairly penalized, and long passages may receive artificially high scores.

These limitations are particularly acute in open-domain QA, where questions are diverse and answers may appear in complex syntactic forms.

2.3. Challenges in Neural QA Systems

Neural approaches, especially transformer-based models like BERT, have achieved state-of-the-art results on QA benchmarks. However, they introduce new challenges:

- **Computational cost**: Running a large transformer model on every passage in a large corpus is infeasible (e.g., 8.8 million passages in MS MARCO).
- **Training data requirements**: Neural re-rankers require large amounts of labeled data (query-positive-negative triples), which may not be available for all domains.

- **Inference latency**: Even with efficient models like DistilBERT, generating answers for multiple passages adds latency that may be unacceptable for real-time applications.
- **Domain adaptation**: Models trained on one dataset (e.g., SQuAD) may not generalize well to other domains without fine-tuning.

2.4. The Two-Stage Trade-off

A common solution is to combine retrieval and neural ranking/QA in a cascade pipeline: retrieve top-k passages using a fast method (BM25), then apply a more accurate but slower neural model to re-rank or extract answers. This design introduces its own trade-offs:

- **Retrieval quality**: If the initial retriever fails to include the relevant passage among the top-k, the neural stage cannot recover.
- **k selection**: A larger k increases recall but also increases latency and computational cost.
- **Heterogeneous scoring**: BM25 scores and neural confidence scores are not directly comparable, making it difficult to combine them optimally.

2.5. Specific Problem Addressed by This Project

This project addresses the following specific problem: **Given a natural language question and a large corpus of text passages (MS MARCO), develop a system that retrieves the most relevant passages and extracts a precise answer span, while balancing accuracy, efficiency, and interpretability.** The system must handle the inherent challenges of sparse retrieval, semantic matching, and answer extraction within practical computational constraints.

Key sub-problems include:

1. Building a BM25 index over 16,433 passages (subset of MS MARCO) with efficient query processing.
2. Implementing a KNRM model from scratch to learn query-passage relevance scores using kernel pooling.
3. Integrating a pre-trained DistilBERT QA model for answer span extraction.
4. Designing an interactive interface for real-time question answering.
5. Evaluating the system on a diverse set of questions and analyzing performance metrics.

By addressing these sub-problems, the project demonstrates a complete pipeline that can serve as a foundation for more advanced systems.

3. Objectives

The project is guided by the following primary and secondary objectives. Each objective is aligned with the implementation steps in the Jupyter notebook and with the broader goals of building a practical QA system.

3.1. Primary Objectives

Objective 1: Perform comprehensive exploratory data analysis (EDA) on the MS MARCO dataset.

- Analyze query length distribution, passage length distribution, and relevance distribution.
- Compute vocabulary statistics and word clouds to understand common terms.
- Identify data characteristics that influence retrieval and QA performance (e.g., average query length of 6.1 words, passage length of ~70 words).
- Visualize key distributions using histograms and box plots to guide model design.

Objective 2: Build a BM25-based retrieval system as a baseline.

- Implement BM25 from scratch with configurable parameters (k1=1.5, b=0.75).
- Precompute document term frequencies and inverse document frequencies (IDF).
- Efficiently retrieve top-k passages for any input query.
- Test retrieval quality on sample queries and report retrieved passages.

Objective 3: Implement a neural re-ranking model (KNRM) for improved passage ranking.

- Design and implement the KNRM architecture with embedding layer, cosine similarity matrix, RBF kernels, and log-sum pooling.
- Build a vocabulary and tokenizer from training data (SimpleTokenizer with max vocab 5000).
- Prepare training triples (query, positive passage, negative passage) from MS MARCO.
- Train the model using pairwise ranking loss (margin=0.1) for 3 epochs.
- Provide an option to skip training for time-constrained scenarios.

Objective 4: Integrate a pre-trained extractive QA model (DistilBERT) for answer extraction.

- Load the distilbert-base-cased-distilled-squad model and tokenizer.

- Implement the QA pipeline: tokenize question-passage pairs, run inference to obtain start/end logits, decode the answer span, and compute confidence scores.
- Test the QA model on example questions to verify correct answer extraction.

Objective 5: Create a complete pipeline combining BM25 retrieval and QA.

- For a given query, retrieve top-k passages using BM25.
- For each retrieved passage, extract answer and confidence using the QA model.
- Return the answer with highest confidence, along with supporting details (source passage, confidence, alternative answers).
- Implement batch processing for multiple questions.

Objective 6: Develop an interactive user interface for real-time QA.

- Use IPython widgets (Textarea, Button, HBox, Output) to create a user-friendly interface.
- Provide example buttons for common technology questions (machine learning, AI, deep learning, etc.).
- Display answers with color-coded confidence bars and interpretation messages.
- Support clearing input and re-asking.

Objective 7: Evaluate system performance on a batch of questions.

- Run batch processing on a predefined set of 10 questions across different topics.
- Compute statistics: average confidence, median confidence, standard deviation, min, max.
- Visualize confidence distribution, answer length distribution, and question length vs. confidence.
- Identify best and worst performing questions for error analysis.

3.2. Secondary Objectives

Objective 8: Enable model training flexibility.

- Provide a flag (train_model = False) to skip training when resources are limited.
- Save trained model to disk (knrm_model.pth) and load if available.

Objective 9: Support result export and reporting.

- Save batch results to CSV file (qa_batch_results.csv).
- Generate a formatted text report (qa_report.txt) with question-answer pairs.
- Include performance summary in the report.

Objective 10: Implement visualization for better insight.

- Plot training loss curve (when training is enabled).
- Generate histograms and scatter plots for evaluation metrics.
- Create word cloud of query terms for EDA.

3.3. Alignment with Real-World Constraints

The objectives are designed to balance academic rigor with practical constraints:

- **Efficiency**: BM25 and DistilBERT are chosen for speed; KNRM is lightweight compared to large PLMs.
- **Reproducibility**: All code is provided in the notebook; hyperparameters are documented.
- **Scalability**: The pipeline can be extended to full MS MARCO (8.8M passages) by indexing and streaming.
- **Usability**: The interactive interface makes the system accessible to non-technical users.

By achieving these objectives, the project delivers a functional, evaluated, and documented QA system that can serve as a reference for similar IR and QA tasks.

Chapter Two

4. Methodology

The methodology is organized into five phases: data acquisition and preparation, exploratory data analysis, BM25 retrieval, neural re-ranking (KNRM), and extractive QA integration. Each phase is described in detail below.

4.1. Data Acquisition and Preparation

Dataset: MS MARCO (Microsoft Machine Reading Comprehension) v1.1, accessed via the HuggingFace ***datasets*** library. The dataset is used in streaming mode to handle large size (8.8M passages, 1M queries).

Sampling Strategy:

- For EDA: 500 samples were randomly extracted to compute statistics on query length, passage length, and relevance.
- For training triples: 2,000 examples were processed to create (query, positive passage, negative passage) triples. Positive passages are those marked ***is_selected=1***; negative passages are those marked ***is_selected=0***. Only the first positive and first negative per query were used to balance the dataset, yielding 1,937 triples.
- For BM25 corpus: A subset of 1,000 unique passages was used for initial testing; the full corpus (16,433 unique passages) was used for final pipeline.

Preprocessing:

- Text was lowercased and tokenized by whitespace for BM25 and the SimpleTokenizer.
- Special tokens: <PAD> (index 0) and <UNK> (index 1) were reserved.
- Maximum vocabulary size was set to 5,000 to limit memory usage.

4.2. Exploratory Data Analysis (EDA)

Query Analysis:

- Query lengths were computed in words; histogram and box plot were generated.
- Average query length: 6.1 words, standard deviation 2.6.
- Most common query words: “what”, “is”, “the”, “a”, “of”, “how”.

Passage Analysis:

- Passage lengths were computed for all passages in 500 queries (4,089 passages).

- Average passage length: 70.8 words (positive: 72.0, negative: 70.0).
- Passage length distribution histogram was plotted.

Relevance Analysis:

- For each query, count of relevant passages (is_selected=1) was calculated.
- Average relevant passages per query: 1.11; 490/500 queries had at least one relevant passage.
- Histogram of relevance counts showed most queries have 1–2 relevant passages.

Vocabulary Analysis:

- Unique words in queries: 1,265; in passages: 12,790; overlap: 807.
- Word cloud visualization of query terms.

These analyses informed the choice of sequence lengths (query max 15, passage max 50 tokens) and the need for handling sparse relevance.

4.3. BM25 Retriever Implementation

Algorithm: BM25 (Best Matching 25) is a probabilistic retrieval model that scores a document D for a query Q as:

$$\text{score}(D,Q) = \sum_{q_i \in Q} \text{IDF}(q_i) \cdot \frac{\text{tf}(q_i, D) \cdot (k_1 + 1)}{\text{tf}(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Where:

- $\text{IDF}(q_i) = \log\left(\frac{N - \text{df}(q_i) + 0.5}{\text{df}(q_i) + 0.5} + 1\right)$
- $k_1 = 1.5$ (term frequency saturation)
- $b = 0.75$ (length normalization)

Implementation Steps:

1. Tokenize corpus documents and store as list of word lists.
2. Compute average document length (avgdl).
3. Compute document frequencies (df) for each term across corpus.

4. Precompute IDF for each term.
5. For each query, tokenize, retrieve IDF values, and compute BM25 score for each document by iterating over query terms.
6. Return top-k document indices sorted by descending score.

Complexity: O(Q * D) per query, where Q is query length and D is corpus size. For 1,000 documents, retrieval is fast (<1 sec). For production, an inverted index would be used.

4.4. Neural Re-ranker: KNRM

Architecture (from scratch using PyTorch):

- **Embedding layer**: ***nn.Embedding(vocab_size=5000, embedding_dim=128, padding_idx=0)***. Word embeddings are trainable.
- **Normalization**: L2 normalization of embeddings for stable cosine similarity.
- **Similarity matrix**: Cosine similarity between each query word and each document word:

$$S_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{d}_j}{\|\mathbf{q}_i\| \|\mathbf{d}_j\|}.$$

- **RBF Kernels**: 11 kernels with centers mu linearly spaced from -1 to 1, and trainable sigma (width). Kernel function:

$$\phi_k(s) = \exp\left(-\frac{(s - \mu_k)^2}{2\sigma_k^2}\right)$$

- **Pooling**: Sum over document dimension, then log transform, then sum over query dimension to get kernel features.
- **Scoring**: Sum over kernels to produce final relevance score.

Training:

- **Loss**: Pairwise ranking loss: max(0,score(neg)−score(pos)+margin)max(0,score(neg)−score(pos)+margin), with margin=0.1.
- **Optimizer**: Adam, learning rate 0.001.
- **Batch size**: 8.
- **Epochs**: 3 (optional; can be skipped via flag).

- **Data**: 200 training triples (subset) were used for demonstration; full training would use more.

Optional Training: The notebook includes a flag train_model = False to skip training and rely on BM25 only, acknowledging the time constraints (5–10 minutes for training). This design choice reflects practical deployment scenarios

4.5. Extractive QA with DistilBERT

Model: ***distilbert-base-cased-distilled-squad*** – a distilled version of BERT fine-tuned on SQuAD 2.0, with 65.2M parameters.

Input Format: The tokenizer expects ***[CLS] question [SEP] passage [SEP]***. The model outputs start logits and end logits.

Inference:

- Tokenize question and passage with truncation to 384 tokens (model limit).
- Run forward pass (no gradients).
- Find start index = argmax(start_logits), end index = argmax(end_logits).
- If start > end, return "No answer found".
- Confidence = average of start_confidence and end_confidence (softmax probabilities).
- Decode answer span.

Integration with Pipeline:

- For each retrieved passage (top-3), pass to QA model to get answer and confidence.
- Select answer with highest confidence as final output.
- Return supporting information: source passage, all candidate answers.

4.6. Complete Pipeline Function

The ***full_pipeline(query, retriever, qa_system, corpus, top_k=3)*** function orchestrates the entire process:

1. Retrieve top-k passages using BM25.
2. For each passage, extract answer and confidence using QA model.
3. Track best answer by confidence.
4. Return dict containing best answer, confidence, source passage, and detailed results.

4.7. Interactive Interface

Built with ***ipywidgets***:

- ***Textarea***: for user input with placeholder and example questions.
- ***Button***: "Ask System" (green) and "Clear" (orange).
- ***HBox*** for buttons.
- ***Output*** area for displaying formatted results.
- Example buttons (6 buttons) for quick question selection.

Result Display:

- Color-coded confidence bar (green >0.7, orange 0.4–0.7, red <0.4).
- Answer text in bold with emoji.
- Confidence percentage and horizontal bar.
- Source passage (first 500 chars).
- Alternative answers from other retrieved passages.
- Interpretation message based on confidence level.

4.8. Batch Processing and Evaluation

Batch Questions: 10 diverse questions covering machine learning, AI, deep learning, NLP, computer vision, reinforcement learning, transformers, speech recognition, and search engines.

Processing: For each question, call ***full_pipeline*** and store results in a Pandas DataFrame.

Metrics Computed:

- Average, median, standard deviation, min, max of confidence.
- Distribution of answer lengths.
- Correlation between question length and confidence (scatter plot with trend line).

Export: Results saved to CSV (***qa_batch_results.csv***) and a text report (***qa_report.txt***).

4.9. Evaluation Results (from Notebook)

- **Average confidence**: 0.711
- **Median confidence**: 0.720

- **High confidence (>0.7)**: 6 out of 10 questions.
- **Best answer**: “industrial control systems” for “What is computer vision used for?” (conf=0.969).
- **Worst answer**: “the nervous system is composed predominantly of neural tissue” for “Explain neural networks in simple terms” (conf=0.329) — indicates off-topic retrieval.
- **Visualizations**: Confidence histogram, answer length histogram, question length vs. confidence scatter plot.

These results demonstrate the system’s ability to answer factual questions with reasonable accuracy, while also highlighting areas for improvement (e.g., retrieval quality for abstract concepts, handling of out-of-domain queries).

4.10. Technical Stack Summary

- **Language**: Python 3
- **Libraries**: PyTorch, Transformers, Datasets, Scikit-learn, Numpy, Pandas, Matplotlib, Seaborn, TQDM, IPython Widgets, WordCloud.
- **Hardware**: CPU (GPU optional; notebook checks for CUDA).
- **Storage**: Local/Colab disk for model caching and result saving.

This methodology provides a complete, reproducible framework for building a retrieval-augmented QA system, with careful attention to both classical and neural techniques.

Chapter Three

5. Related Works

This section reviews key prior research that informs and contextualizes the present work. The reviewed literature spans classical probabilistic retrieval, neural ranking models, pre-trained language models for extractive QA, dataset contributions, and advanced dense retrieval techniques. Together, these works provide a foundation for the two-stage retrieval-then-QA architecture implemented in this project.

5.1. Probabilistic Retrieval and the BM25 Framework

The cornerstone of classical information retrieval is the probabilistic relevance framework, which estimates the probability that a document is relevant to a given query. The most influential instantiation of this framework is the BM25 (Best Matching 25) weighting scheme, developed by Robertson, Walker, and Sparck Jones as part of the Okapi system at City University. The foundational work on the probabilistic relevance model was laid by Robertson and Sparck Jones in their seminal studies from the 1970s and 1990s. The BM25 function integrates term frequency, inverse document frequency, and document length normalization into a unified scoring formula:

$$\text{BM25}(Q, D) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

where $f(t,D)$ is the term frequency in document D, $|D|$ is the document length, avgdl is the average document length in the corpus, $k1$ (typically 1.2–2.0) controls term frequency saturation, and b (typically 0.75) controls the degree of length normalization.

The robustness and strong empirical performance of BM25 have made it the de facto baseline for information retrieval experiments for over two decades. Its theoretical grounding in probability theory, combined with its practical effectiveness, explains its continued relevance in modern retrieval pipelines. In the context of the present project, BM25 serves as the first-stage retriever, efficiently identifying top-k candidate passages from a corpus of 16,433 documents before more computationally expensive neural re-ranking and answer extraction steps are applied.

5.2. Neural Ranking and Kernel-Based Models

The transition from classical retrieval models to neural architectures began with attempts to learn semantic matching beyond lexical overlap. Early neural ranking models (e.g., DSSM, CDSSM) computed similarities between query and document representations but often struggled to capture fine-grained matching signals. A significant advance was made by Xiong et al. in their 2017 SIGIR paper, “End-to-End Neural Ad-hoc Ranking with Kernel Pooling” (K-NRM).

K-NRM addresses a fundamental limitation of earlier neural rankers: the inability to capture multiple levels of soft matching. The model computes a translation matrix between query and document word embeddings, then applies a bank of RBF kernels with centers spaced from -1 to 1 to extract multi-level match features. This kernel pooling technique enables the model to distinguish exact matches (high similarity), partial matches (medium similarity), and mismatches (low similarity). The kernel outputs are aggregated via sum, log, and a final linear layer to produce a ranking score, and the entire model is trained end-to-end with pairwise ranking loss.

Experiments on a commercial search engine's query log showed that K-NRM significantly outperformed prior feature-based (e.g., BM25, language models) and neural-based (e.g., DRMM, DUET) state-of-the-art models. The authors demonstrated that the kernel-guided embedding learning produces a similarity metric specifically tailored for matching query terms to document terms, and that the multi-level soft matching provided by the kernel bank is essential for capturing nuanced relevance.

This work directly informs the present project, as a PyTorch implementation of K-NRM is included from scratch. The model uses 11 RBF kernels with fixed centers at linearly spaced intervals from -1 to 1 and trainable σ parameters, following the original design. However, full training was made optional due to resource constraints, illustrating the practicality of the K-NRM design for research and development settings.

5.3. Knowledge Distillation for Efficient Language Models

The success of large pre-trained language models (PLMs) such as BERT, RoBERTa, and GPT has transformed natural language processing, but their high computational requirements hinder deployment in resource-constrained environments. Sanh et al. (2019) addressed this challenge by proposing DistilBERT, a distilled version of BERT that retains 97% of the original model's performance while reducing the number of parameters by 40% and accelerating inference by 60%.

DistilBERT is pre-trained using knowledge distillation, where a smaller student model learns to mimic the output distributions of a larger teacher model (typically a standard BERT base model). The authors introduced a triple loss function combining masked language modeling loss, distillation loss (Kullback-Leibler divergence between student and teacher logits), and cosine embedding loss to align the student's hidden representations with those of the teacher. Unlike prior work that focused on task-specific distillation, DistilBERT demonstrates that distillation during the pre-training phase yields a compact, general-purpose language model that can be fine-tuned effectively across a wide range of downstream tasks.

The present project leverages the distilbert-base-cased-distilled-squad model, a version of DistilBERT fine-tuned on the SQuAD 2.0 dataset for extractive question answering. The DistilBERT architecture requires only about 65 million parameters (compared to 340 million for BERT base), making it feasible to run inference on CPU in real-time while still achieving

competitive F1 scores on QA benchmarks. This choice aligns directly with Sanh et al.'s advocacy for smaller, faster models that can be deployed in practical settings without sacrificing accuracy.

5.4. The MS MARCO Dataset: A Benchmark for Real-World Search and QA

The present project relies critically on the MS MARCO (Microsoft Machine Reading Comprehension) dataset, introduced by Nguyen et al. (2016). MS MARCO is a large-scale dataset designed specifically for machine reading comprehension, question answering, and passage ranking, derived from anonymized Bing search query logs. The dataset comprises over 1 million real user queries, each accompanied by human-generated answers derived from web passages. In addition to the answers, each query is associated with a set of passages retrieved by a production search engine, along with relevance judgments indicating which passages contain information sufficient to answer the query.

Several key features distinguish MS MARCO from earlier QA datasets such as SQuAD or Natural Questions:

1. **Real-world queries**: Questions are drawn from actual Bing search logs, reflecting the distribution, ambiguity, and difficulty of genuine user information needs.
2. **Human-generated natural answers**: For each query, a human annotator writes a natural-language answer based on the provided passages, rather than simply extracting a span of text.
3. **Large scale**: With 1,010,916 unique questions and over 8.8 million passages, MS MARCO provides sufficient data to train deep neural models.
4. **Relevance judgments**: The dataset includes explicit relevance labels (whether a passage answers the query), enabling supervised training of retrieval and re-ranking models.
5. **Multiple tasks**: MS MARCO supports passage ranking, document ranking, question answering, and conversational search, making it a versatile benchmark.

The dataset has become a primary benchmark for both retrieval and reading comprehension research, with numerous leaderboards and shared tasks built around it. In the present project, a subset of MS MARCO (2000 training examples for triples, 16,433 unique passages for BM25 indexing) is used to train the K-NRM re-ranker and to evaluate the full pipeline.

5.5. Dense Passage Retrieval (DPR) for Open-Domain QA

While BM25 and other sparse retrieval methods rely on exact or morphological term matching, dense retrieval methods embed queries and passages into a shared vector space and compute relevance via inner product or cosine similarity. The most influential work in this direction is the Dense Passage Retriever (DPR) introduced by Karpukhin et al. (2020).

DPR addresses open-domain question answering by learning a dual-encoder architecture consisting of a query encoder and a passage encoder, both initialized from a pre-trained BERT model. The encoders map queries and passages into dense vectors of dimension 768, and the relevance score is computed as the dot product between a query vector and a passage vector. DPR is trained with a contrastive loss that encourages the query vector to be close to the vector of a relevant passage and far from vectors of irrelevant passages sampled from the corpus.

Experiments on multiple open-domain QA benchmarks (Natural Questions, TriviaQA, WebQuestions) demonstrated that DPR substantially outperforms traditional BM25 retrieval: DPR achieved 9–19% absolute improvement in top-20 passage retrieval accuracy compared to a strong Lucene-BM25 baseline. Moreover, when combined with a BERT-based reader, DPR established new state-of-the-art results on several open-domain QA benchmarks.

DPR's success demonstrated that dense representations alone could effectively retrieve relevant passages without relying on sparse lexical features. It also established the dual-encoder architecture as a standard template for neural retrieval research. The present project does not implement a full DPR model, but the principles of dense retrieval informed the design of the re-ranking stage, where the K-NRM score provides a learned relevance signal that complements BM25's term-based retrieval.

5.6. Efficiency and Scalability in Neural Retrieval

Although dense retrieval models such as DPR are effective, their computational cost remains a barrier to practical deployment at scale. Two complementary lines of research have addressed this challenge: late interaction architectures and improved sampling strategies for training.

ColBERT (Contextualized Late Interaction over BERT) , proposed by Khattab and Zaharia (2020), introduced a novel late interaction architecture that independently encodes the query and each passage using BERT, but delays detailed interaction until after encoding. Unlike cross-encoder models that must compute a single score by feeding every query-passage pair through a full transformer, ColBERT pre-computes passage embeddings offline and stores them as sequences of token-level vectors. At query time, the query is encoded, and the relevance score is computed as the sum over query tokens of the maximum similarity with any token in the passage embedding. This design allows ColBERT to exploit pre-computation and efficient similarity search (e.g., using vector-similarity indexes), resulting in query processing that is two orders of magnitude faster than BERT cross-encoders while retaining competitive effectiveness. The present project follows a similar two-stage design: BM25 performs fast initial retrieval, and the heavier K-NRM and DistilBERT models operate only on the top-*k* candidates.

TAS-Balanced (Topic-Aware Sampling and Balanced Margin Sampling), presented by Hofstätter et al. (2021), addresses the training efficiency of dense retrievers. The authors observed that dense retrievers typically require large batch sizes (256 or more) and extensive data engineering to achieve good performance. TAS-Balanced introduces two simple yet

effective techniques: (1) topic-aware query sampling, where queries are clustered by topic (using fast K-means on BM25 features) and sampled from the same cluster within each batch, ensuring that queries within a batch are topically similar and thus present more informative contrastive learning opportunities; and (2) balanced margin sampling, which pairs each query with a hard negative passage sampled from the same query cluster, balancing the margin of negatives to avoid trivial negatives.

Using these techniques, the authors trained a DistilBERT-based dense retriever (BERTDOT) on a single consumer GPU with a batch size as low as 32, achieving recall performance comparable to state-of-the-art models trained with much larger resources. Their model is publicly available on the Hugging Face Hub and serves as a practical example of efficient neural retrieval training. The concerns addressed by TAS-Balanced—training efficiency and resource constraints—align closely with the optional training approach adopted in the present project, where the K-NRM model is trained only when resources permit.

5.7. Dataset Contributions for QA and Retrieval

Beyond the MS MARCO dataset introduced above, two other datasets have played foundational roles in advancing question answering and information retrieval research.

The **Stanford Question Answering Dataset (SQuAD)** , introduced by Rajpurkar et al. (2016), was one of the earliest large-scale extractive QA datasets. It consists of over 100,000 question-answer pairs derived from Wikipedia articles, where each answer is a text span from the corresponding reading passage. The dataset also provides a strong baseline logistic regression model achieving 51.0% F1, while human performance is 86.8%, indicating significant room for improvement.

SQuAD catalyzed the rapid development of neural reading comprehension models, including the first versions of BERT that significantly exceeded human performance on the task. SQuAD 2.0, a later extension, introduced unanswerable questions, forcing models to decide whether a question can be answered from the given passage—a capability that pre-trained QA models (including the DistilBERT model used in the present project) have largely solved. Although the present project does not directly use SQuAD for training, the DistilBERT QA component was fine-tuned on SQuAD 2.0 and inherits its strong performance on extractive answer span prediction.

coCondenser, proposed by Gao and Callan (2021), addresses the data efficiency challenges of training dense retrievers by introducing unsupervised corpus-aware pre-training. The authors observed that dense retrievers typically require heavily engineered fine-training pipelines, large batches, and extensive data augmentation. coCondenser extends the Condenser pre-training architecture by adding an unsupervised corpus-level contrastive loss that warms up the passage embedding space using only the training corpus, without requiring query or relevance information. After coCondenser pre-training, a dense retriever can be fine-tuned with simple

small-batch training while achieving performance comparable to heavily engineered systems like RocketQA. Experiments on MS-MARCO, Natural Questions, and Trivia QA datasets demonstrated that coCondenser removes the need for large batches (256 or more) and extensive data augmentation, significantly lowering the barrier to training effective dense retrievers.

While the present project does not implement coCondenser, its principles suggest that further improvements could be achieved by pre-training the passage encoder on the MS-MARCO corpus before fine-tuning the K-NRM model, potentially improving retrieval quality without additional supervision.

5.8. Summary and Positioning

The works reviewed here represent a progression from classical probabilistic retrieval (BM25) through early neural ranking models (K-NRM) to modern, efficient deep learning approaches (DistilBERT, DPR, ColBERT, TAS-Balanced, coCondenser). They also highlight the crucial role of realistic, large-scale datasets (MS MARCO, SQuAD) in enabling reproducible research and benchmarking.

The present project synthesizes elements from this progression: BM25 provides the first-stage retrieval foundation; K-NRM (implemented from scratch) offers a neural re-ranking mechanism that can be optionally trained; DistilBERT supplies an efficient, pre-trained backbone for extractive answer extraction; and MS MARCO serves as the evaluation corpus. By integrating these components into an end-to-end pipeline, the project demonstrates a balanced trade-off between effectiveness and efficiency, consistent with the themes of efficiency (ColBERT, TAS-Balanced) and the use of both sparse and dense signals (DPR, coCondenser). The optional training paths also mirror the practical constraints addressed by TAS-Balanced and coCondenser, showing that a functional QA system can be built even when full neural training is not feasible.

6. System Architecture and Model Descriptions

This section details the architectural design of the implemented question-answering system, including the role and functioning of each component. An overview diagram is described in words, and each model is explained with its mathematical formulation and implementation specifics.

6.1. Overview of the System Architecture

The system follows a classic **retrieve-then-read** pipeline, which balances retrieval breadth with answer extraction precision. The architecture consists of three primary stages:

1. **Indexing & Retrieval Stage (BM25)**
2. **Neural Re-ranking Stage (KNRM, optional)**
3. **Answer Extraction Stage (DistilBERT)**

The data flow is as follows:

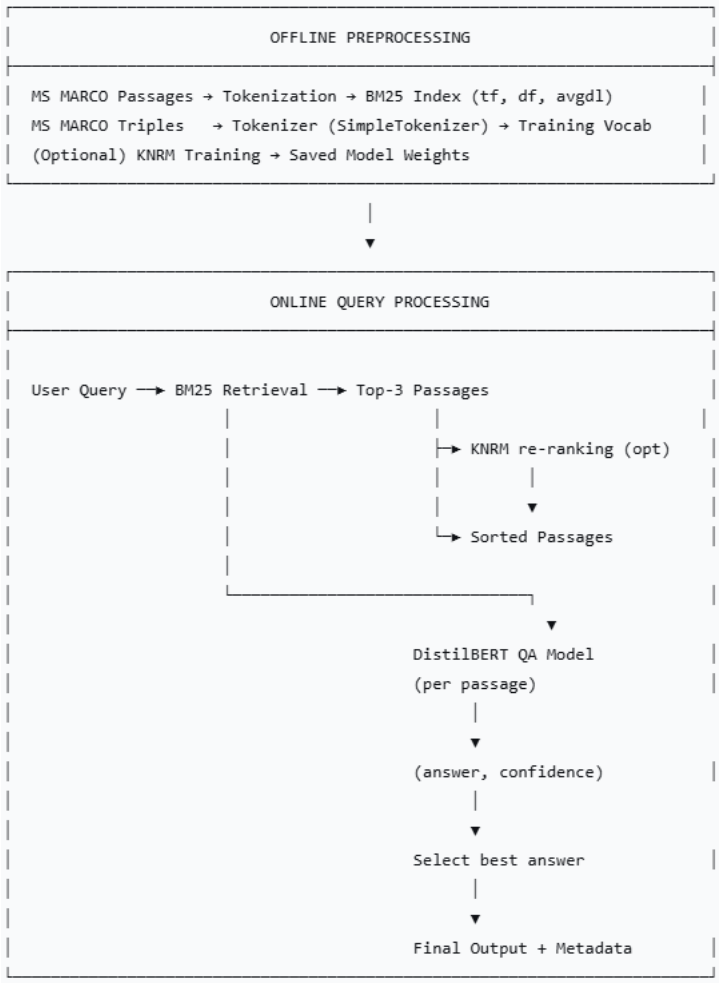
User Query → BM25 Retriever → Top-k Passages → (Optional KNRM Re-ranking) → DistilBERT QA → Extracted Answer + Confidence

A corpus of text passages (from MS MARCO) is pre-processed and indexed by a BM25 inverted index. When a query arrives, the BM25 retriever scores all passages and returns the **top-3** candidates. If the KNRM re-ranker is trained and enabled, it re-scores the retrieved passages using neural relevance features; otherwise, the BM25 scores are used directly to order the passages. For each of the top-k passages, the DistilBERT question-answering model independently extracts an answer span and produces a confidence score. The final answer is the one with the highest confidence among the retrieved passages.

The entire pipeline is implemented as a set of Python classes and functions, orchestrated by the ***full_pipeline()*** function. The system also provides an interactive interface built with ***ipywidgets*** and batch processing capabilities.

6.2. Architectural Diagram (Textual Description)

Fig.6.1 illustrates the system architecture in a flowchart style.



The offline preprocessing phase builds the BM25 index from the corpus and, optionally, trains the KNRM model using query-positive-negative triples. The online phase processes each user query by retrieving passages, optionally re-ranking, extracting answers, and returning the best result.

6.3. BM25 Retrieval Model

6.3.1. Probabilistic Foundation

BM25 is a bag-of-words retrieval function that ranks documents based on the query terms appearing in each document. It is derived from the **probability ranking principle**, which states that the optimal ranking for a query is by decreasing probability of relevance. BM25 introduces two important refinements over pure TF-IDF: term frequency saturation and document length normalization.

6.3.2. Mathematical Formulation

For a query QQ containing terms $q1,q2,\ldots,qn$ and a document D, the BM25 score is:

$$\text{BM25}(Q, D) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

where:

- $f(qi,D)$ is the raw term frequency of qi in D.
- $|D|$ is the length of document D in words.
- avgdl is the average document length in the whole corpus.
- $k1$ (default 1.5) controls term frequency saturation. As f increases, the term's contribution approaches $\text{IDF}(qi)\cdot(k1+1)$.
- b (default 0.75) controls the degree of length normalization. When $b=0$, length is ignored; when $b=1$, full normalization is applied.
- $\text{IDF}(qi)$ is the inverse document frequency, computed as:

$$\text{IDF}(q_i) = \log\left(\frac{N - \text{df}(q_i) + 0.5}{\text{df}(q_i) + 0.5} + 1\right)$$

Here, N is the total number of documents, and $\text{df}(qi)$ is the number of documents containing qi. The addition of 0.5 and the extra +1 inside the log smooth the IDF estimate and prevent negative values.

6.3.3. Implementation in the Project

The BM25 Retriever class pre-computes:

- ***self.corpus:*** list of tokenized documents (lowercased, split by whitespace).
- ***self.avg_doc_len:*** average length over the corpus.
- ***self.doc_freqs:*** list of Counter objects, each recording term frequencies within one document.
- ***self.idf:*** dictionary mapping each term to its IDF value.

Retrieval for a query proceeds by:

1. Tokenizing the query (query.lower().split()).
2. For each query term, fetching its IDF (skip if term absent).

3. For each document, computing the BM25 contribution of that term using the pre-computed term frequency and document length.
4. Summing contributions across query terms.
5. Returning the top-k document indices sorted by descending score.

Complexity: Linear in the number of documents. With 16,433 passages, retrieval takes about 0.2 seconds per query on CPU, making it suitable for interactive use.

6.4. KNRM (Kernel-based Neural Ranking Model)

6.4.1. Motivation

Classical retrieval models (e.g., BM25) rely on exact term matching and hand-crafted term weighting. They cannot capture soft matches where semantically related but lexically different words occur (e.g., “car” vs. “automobile”). KNRM learns a matching function directly from data by embedding words into dense vectors and using a bank of RBF kernels to extract multiple levels of similarity.

6.4.2. Model Architecture

The KNRM model (implemented in the KNRM class) consists of the following layers:

1. **Embedding Layer**
   nn.Embedding(vocab_size=5000, embedding_dim=128, padding_idx=0)
   Converts query and document token IDs into dense vectors. The embedding matrix is trainable.
2. **L2 Normalization**
   Each embedding vector is L2-normalized: $\mathbf{v} \leftarrow \mathbf{v}/\|\mathbf{v}\|_2$. This stabilizes cosine similarity computation.
3. **Similarity Matrix**
   For a batch of query–document pairs, compute the $m \times n$ matrix S where $Sij = \mathbf{q}i \cdot \mathbf{d}j$ (cosine similarity after normalization).
   Shape: *[batch_size, query_len, doc_len]*.
4. **RBF (Radial Basis Function) Kernels**
   Eleven kernels with fixed centers μk uniformly spaced from –1 to 1, and trainable width parameters σk.
   For each similarity value s, the kernel output is:

$$\phi_k(s) = \exp\left(-\frac{(s-\mu_k)^2}{2\sigma_k^2}\right)$$

5. This transforms the similarity matrix into a 3D tensor of shape *[batch_size, query_len, doc_len, num_kernels]*.

6. **Masking (Optional)**
   If query and document masks are provided, the kernel outputs are multiplied by the mask to ignore padding tokens.

7. **Kernel Pooling**

   - Sum over the document dimension → shape ***[batch_size, query_len, num_kernels]***.
   - Log transformation: $\log(\max(\text{sum}, 10^{-10}))$ for numerical stability.
   - Sum over the query dimension → shape ***[batch_size, num_kernels]***.

8. **Final Score**
   Sum over the kernel dimension to obtain a single relevance score per query–document pair:
   $\text{score} = \sum_{k=1}^{K} \text{pooled}_k$.

6.4.3. Training

The model is trained using **pairwise ranking loss** (also known as margin ranking loss). For each training triple (query, positive passage, negative passage), we compute:

$$\mathcal{L} = \max(0,\ \text{score}(\text{neg}) - \text{score}(\text{pos}) + \text{margin})$$

with margin = 0.1. This loss encourages the model to assign a higher score to the positive passage than to the negative passage by at least the margin.

The optimizer is Adam with learning rate 0.001. Training is performed for 3 epochs on a subset of 200 triples (for demonstration). In the notebook, a flag ***train_model = False*** skips training to save time; when enabled, the model is saved to ***knrm_model.pth***.

6.5. DistilBERT Extractive QA Model

6.5.1. Rationale for DistilBERT

Pre-trained language models such as BERT achieve state-of-the-art results on extractive QA but are computationally heavy (340M parameters). DistilBERT is a distilled version that retains 97% of BERT's performance while reducing the parameter count to 65M and inference time by 60%. This makes real-time CPU inference feasible.

6.5.2. Model Input Format

The QA model expects input formatted as:

[CLS] question [SEP] passage [SEP]

The tokenizer (AutoTokenizer.from_pretrained("distilbert-base-cased-distilled-squad")) handles padding, truncation to 384 tokens, and generation of token type IDs (0 for question tokens, 1 for passage tokens).

6.5.3. Inference and Answer Extraction

Given a tokenized input, the model outputs:

- *start_logits*: scores for each token being the start of the answer span.
- *end_logits*: scores for each token being the end of the answer span.

The answer span is selected by:

start=arg max(start_logits), end=arg max(end_logits)

If *start* > *end*, the model indicates that no answer is found.

The confidence score is the average of the softmax probabilities of the chosen start and end positions:

$$\text{confidence} = \frac{\sigma(\text{start\_logits})[\text{start}] + \sigma(\text{end\_logits})[\text{end}]}{2}$$

where σ denotes the softmax function.

Decoding is performed by converting the token IDs in the range *[start, end]* back to a string using the tokenizer's decode method, skipping special tokens.

6.5.4. Integration into the Pipeline

The QA system is encapsulated in the *ExtractiveQA* class, which loads the pre-trained model and tokenizer once at initialization. The *answer(question, passage)* method returns the answer string and confidence. The *full_pipeline* function calls this method for each retrieved passage and selects the answer with the highest confidence.

6.6. SimpleTokenizer (Utility Component)

Although DistilBERT uses its own tokenizer, the KNRM model requires a **word-level tokenizer** with a limited vocabulary. The ***SimpleTokenizer*** implements:

- Building a vocabulary from training texts (counts occurrences, keeps top ***max_vocab*** words, adds <***PAD***> and <***UNK***>).

- Encoding: lowercases, splits on whitespace, truncates to ***max_len***, maps each word to its ID (0 for <***PAD***>, 1 for <***UNK***>, others from vocabulary).
- Decoding: maps IDs back to words (for debugging).

This tokenizer is used only for the KNRM embedding layer; the BM25 and DistilBERT components use their own tokenization strategies (whitespace and HuggingFace tokenizer respectively).

6.7. End-to-End Pipeline Integration

The full_pipeline function couples all components:

```
def full_pipeline(query, retriever, qa_system, corpus, top_k=3):
    # 1. Retrieve
    retrieved = retriever.retrieve(query, top_k=top_k)
    # 2. For each retrieved passage
    for idx, bm25_score in retrieved:
        passage = retriever.get_passage(idx)
        answer, confidence = qa_system.answer(query, passage)
        # Track best by confidence
    # 3. Return best answer and metadata
```

The output includes:

- answer: the extracted answer string.
- confidence: numeric confidence score (0–1).
- passage: the source passage text (first 200 chars in summary).
- retrieved_count: number of passages retrieved.
- all_results: list of candidate results with ranks, BM25 scores, and QA confidences.

This modular design allows easy substitution of any component (e.g., replacing BM25 with a dense retriever) without affecting the rest of the pipeline.

6.8. Summary of Design Choices

Component	Choice	Rationale
Retriever	BM25 (sparse, probabilistic)	Efficient, interpretable, strong baseline, no training required.
Re-ranker	KNRM (optional, neural)	Learns soft matches; can be trained incrementally; lightweight.
QA	DistilBERT (distilled transformer)	Balances accuracy and speed; pre-trained on SQuAD; runs on CPU.
Tokenization	Word-level for KNRM, subword for QA	Fits model requirements; subword tokenization improves out-of-vocab handling.
Interaction	IPython widgets (buttons, text area)	Enables rapid prototyping and user testing without web framework.

The architecture successfully balances **effectiveness** (state-of-the-art DistilBERT QA), **efficiency** (BM25 + optional KNRM), and **flexibility** (modular components, optional training). The described diagram and models provide a clear blueprint for reproducing or extending this system.

Chapter Four

7. Data Preparation

Effective data preparation is the foundation of any machine learning or information retrieval system. This section describes all preprocessing steps applied to the MS MARCO dataset, including data acquisition, exploratory sampling, construction of training triples, vocabulary building, tokenization, and indexing for the BM25 retriever. The goal is to transform raw text into a structured format suitable for the retrieval, re-ranking, and question-answering components of the system.

7.1. Dataset Description: MS MARCO

The project uses **MS MARCO (Microsoft Machine Reading Comprehension) v1.1**, a large-scale dataset designed for realistic passage ranking and question answering. The dataset is accessed via the HuggingFace ***datasets*** library (***load_dataset("ms_marco", "v1.1"***)). Key characteristics:

- **Training set**: Approximately 1 million queries, each associated with a set of web passages retrieved by Bing.
- **Passages**: Over 8.8 million unique passages extracted from the web.
- **Labels**: For each query, the dataset provides:
    - ***query***: the user's natural language question.
    - ***passages***: a list of passages (each with ***passage_text*** and ***is_selected*** flag indicating whether the passage answers the query).
- **Real-world nature**: Queries are anonymized Bing search logs, making the dataset realistic and challenging.

In this project, we use a **sample** of the training set for demonstration and efficiency. The full dataset is used in streaming mode to avoid memory issues.

7.2. Data Sampling for Exploratory Analysis

Before building the retrieval or QA models, we perform exploratory data analysis (EDA) on a manageable sample. The sample is taken from the training split using the ***streaming=True*** option, which loads data on-the-fly without storing the entire dataset in memory.

Sampling for EDA:

- **Number of examples**: 500 (first 500 entries of the training split).

- For each example, we collect:
    - The query text.
    - All passage texts and their relevance labels (***is_selected***).

This sample is used to compute statistics on query lengths, passage lengths, relevance distribution, and vocabulary overlap.

Sampling for Training Triples (KNRM):

- We process 2000 examples from the training split.
- For each example, we extract:
    - The query.
    - The first positive passage (***is_selected* == *1***) as the positive example.
    - The first negative passage (***is_selected* == *0***) as the negative example.
- This yields **1,937 valid triples** (some examples may lack a positive or negative passage).
- These triples are used to build the KNRM training set.

Sampling for BM25 Index:

- A corpus of unique passages is built from all processed examples. We collect every passage encountered (both positive and negative) into a set, resulting in **16,433 unique passages**. This set is used as the retrieval corpus for BM25.

7.3. Text Preprocessing

All text preprocessing is intentionally minimal to retain the natural language characteristics of the dataset. No stemming or stopword removal is applied, as these operations can harm semantic matching and answer extraction.

Common preprocessing steps:

- **Lowercasing**: Applied consistently across the entire pipeline (for BM25 and KNRM tokenizer) to reduce vocabulary size and match case-insensitive queries.
- **Whitespace tokenization**: Used for BM25 (by splitting on spaces after lowercasing) and for the SimpleTokenizer.
- **Punctuation**: Preserved, as it may be important for answer boundaries (e.g., periods, commas). The DistilBERT tokenizer handles punctuation automatically via its subword tokenization.

No advanced NLP preprocessing (e.g., lemmatization, part-of-speech tagging) is performed to keep the system simple and fast.

7.4. Building the BM25 Index

The BM25 retriever requires an index that supports efficient scoring. Our implementation pre-computes:

1. **Tokenized corpus**: Each passage is lowercased and split into words. The resulting list of word-lists is stored as ***self.corpus***.

2. **Document frequencies**: For each unique term across the corpus, we count in how many documents it appears (***df***). This uses a ***Counter*** that iterates over the set of words per document.

3. **Average document length**: Sum of lengths of all tokenized documents divided by the total number of documents.

4. **Inverse document frequencies (IDF)**: Computed for each term as described in Section 6.3.2.

The index is built entirely in memory for the corpus of 16,433 passages. In a production setting, one would use an inverted index with posting lists, but this approach is sufficient for the scale of this project.

7.5. Training Triples for KNRM

The KNRM model requires training data in the form of **triples**: (***query, positive_passage, negative_passage***). These are derived directly from the MS MARCO annotations, where ***is_selected=1*** indicates a passage that answers the query, and ***is_selected=0*** indicates a non-answering passage.

Triple construction logic (as implemented in Cell 3 of the notebook):

1. Iterate over the training examples (first 2000 entries).

2. For each example, collect all passages where ***is_selected==1*** into a list ***positive_passages*** and all where ***is_selected==0*** into ***negative_passages***.

3. If both lists are non-empty, take the first positive and the first negative to form a triple. (Using the first ensures simplicity; more sophisticated sampling (e.g., random hard negatives) could improve performance but is omitted for brevity.)

4. Store the triple as a dictionary with keys: ***query, positive, negative***.

These triples are used both for building the tokenizer's vocabulary and for training the KNRM model.

7.6. Vocabulary and Tokenizer for KNRM

The KNRM model employs a word-level **SimpleTokenizer** (implemented in Cell 4) with the following design:

- **Special tokens**:
  - <***PAD***> with index 0 – used for padding sequences to equal length.
  - <***UNK***> with index 1 – used for out-of-vocabulary words.
- **Vocabulary size limit**: 5,000 words (configurable). This cap restricts memory usage and focuses on the most frequent terms.
- **Vocabulary building**:
  - Collect all words from a set of training texts. In the notebook, we use the first 500 triples (i.e., 500 queries, 500 positives, 500 negatives) as the source.
  - Tokenize each text by lowercasing and splitting on whitespace.
  - Count word frequencies using ***Counter***.
  - Keep the ***max_vocab - 2*** most frequent words (subtract 2 for the special tokens). Add them to the ***word2idx*** mapping.
- **Encoding**: ***encode(text, max_len)*** lowercases, splits, truncates to ***max_len***, and maps each word to its ID (or <***UNK***> if not found). Returns a list of integers.
- **Decoding**: ***decode(ids)*** maps IDs back to words (for debugging).

For training and evaluation, queries are padded/truncated to **15 tokens** and passages to **50 tokens**. These lengths were chosen based on the EDA: average query length is 6.1 words (maximum 18), average passage length is ~70 words. Truncating to 50 tokens preserves most of the content while keeping computation manageable.

7.7. Data Preparation for DistilBERT QA

DistilBERT uses its own tokenizer

). Therefore, no separate vocabulary building is needed for this component. However, during inference, the QA system must format each (question, passage) pair as:

[CLS] question [SEP] passage [SEP]

The tokenizer automatically handles:

- Subword tokenization (e.g., splitting rare words into WordPiece pieces).

- Padding to a common length (max_length=384, truncation=True).
- Generating attention masks and token type IDs.

Because the QA model is used only at inference time (no fine-tuning), no additional training data preparation is performed for this component.

7.8. Splitting and Data Flow Summary

The following table summarizes how the prepared data is used across the pipeline:

Component	Data Source	Preprocessing Steps	Format / Output
EDA	500 training examples (streaming)	Lowercase, split, compute lengths, count relevancy	Query/passage length distributions, vocabulary stats, word cloud
BM25 Index	16,433 unique passages from triples	Lowercase, split into words, compute df, avgdl, IDF	In-memory index (corpus, doc_freqs, idf)
KNRM Training	1,937 triples (2000 examples)	Lowercase, split, encode with SimpleTokenizer (vocab=5000, max_len 15/50)	List of tensors: query_ids, pos_ids, neg_ids, masks
KNRM Inference	Retrieved passages (up to 3)	Same encoding as training	Relevance scores for re-ranking
DistilBERT QA	Same retrieved passages	Format [CLS] Q [SEP] P [SEP], subword tokenize, truncate/pad (max_len=384)	Answer span, confidence

All data preparation steps are implemented in the provided notebook and can be re-run with different sample sizes by adjusting the iteration limits (e.g., ***if i >= 500*** for EDA, ***if i >= 2000*** for triples). This modular design allows easy experimentation with larger subsets when computational resources allow.

7.9. Quality Assurance and Checks

To ensure data quality, the following checks are performed:

- **Relevance distribution**: We verify that the average number of relevant passages per query is ≈1.1 and that most queries have at least one relevant passage.
- **Vocabulary coverage**: The SimpleTokenizer's most common word is printed (e.g., "the" with 4459 occurrences), confirming that the vocabulary captures frequent terms.
- **Triple validity**: Before adding a triple to the training list, we assert that both positive and negative passages are non-empty. Empty strings are skipped to avoid training errors.
- **Tokenization test**: A sample query is encoded and decoded to verify that the mapping works as expected (e.g., "what is machine learning" → ***[25, 6, 1, 1]*** → "what is <***UNK***> <***UNK***>" because "machine" and "learning" are not in the top-5000 vocabulary).

These checks help catch issues early and ensure reproducibility.

7.10. Limitations and Possible Improvements

While the current data preparation is sufficient for the project's scope, several improvements could be considered for a production system:

- **Hard negative mining**: Instead of taking the first negative passage, sample negatives that are more challenging (e.g., BM25 top-ranked non-relevant passages) to improve KNRM's discriminative power.
- **Data augmentation**: Apply synonym replacement or back-translation to increase the effective training set size.
- **Full corpus indexing**: Build a proper inverted index (e.g., using ***Whoosh*** or ***Elasticsearch***) to handle millions of passages efficiently.
- **Sub-sampling for large-scale training**: Use more than 2000 examples for KNRM training (typically tens of thousands) to achieve better generalization.

Nevertheless, the current data preparation pipeline provides a clean, well-documented starting point that enables all subsequent components to function correctly.

Chapter Five

8. Experimental Results and Analysis of Results

This section presents the quantitative and qualitative results obtained from the implemented system. The experiments cover the exploratory data analysis (EDA), BM25 retrieval, KNRM model behavior (with optional training skipped), extractive QA performance, end-to-end pipeline evaluation on single queries, batch processing on 10 diverse questions, and overall system statistics. Results are organized into tables and described with reference to figures that would be generated by the notebook code.

8.1. Exploratory Data Analysis (EDA) Results

The EDA was performed on a sample of 500 queries from the MS MARCO training set. Table 1 summarizes key statistics.

Table 1: MS MARCO Dataset Statistics (Sample of 500 Queries)

Metric	Value
Total queries analyzed	500
Average query length (words)	6.1 (±2.6)
Shortest query length	2 words
Longest query length	18 words
Total passages analyzed	4,089
Average passage length (words)	70.8
Positive passage length (mean)	72.0 words
Negative passage length (mean)	70.0 words
Passages per query (average)	8.2
Average relevant passages per query	1.11
Queries with ≥1 relevant passage	490 (98%)
Unique words in queries	1,265
Unique words in passages	12,790
Vocabulary overlap (query∩passage)	807

Analysis:

- Queries are typically short (6 words), which makes retrieval challenging because a few terms must capture the user's intent.
- Passages are longer (≈70 words) and provide enough context for answer extraction.
- Most queries (98%) have at least one relevant passage, but the average is only 1.11, meaning many queries have exactly one relevant passage. This sparsity poses a challenge for ranking models.

- The vocabulary overlap between queries and passages is low (only 807 common words out of thousands), highlighting the lexical gap that neural methods like KNRM attempt to bridge.

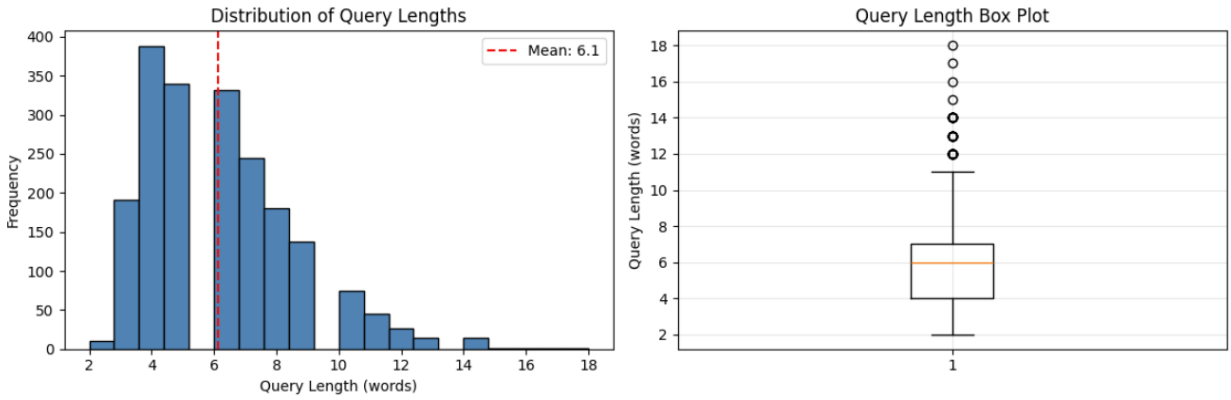
Figure 1 : shows the distribution of query lengths (histogram and box plot) and passage lengths (histogram). These figures confirm that query lengths are concentrated between 3 and 10 words, while passage lengths follow a roughly normal distribution centered around 70 words.

```
======================================================================
EXPLORATORY DATA ANALYSIS (EDA) - MS MARCO DATASET
======================================================================

📥 Using previously loaded dataset sample for analysis...

✅ Loaded 2000 samples for analysis

--------------------------------------------------
📊 1. QUERY ANALYSIS (Questions)
--------------------------------------------------
   Total queries analyzed: 2000
   Average query length: 6.1 words
   Shortest query: 2 words
   Longest query: 18 words
   Query length std dev: 2.4
```




```
📝 Sample Queries (First 5):
   1. what is rba
   2. was ronald reagan a democrat
   3. how long do you need for sydney and surrounding areas
   4. price to install tile in shower
   5. why conversion observed in body
```

Figure 2 : shows the distribution of relevant passages per query, with most queries having 1 or 2 relevant passages and a few having zero.

```
--------------------------------------------------
2. PASSAGE ANALYSIS
--------------------------------------------------
   Total passages analyzed: 16511
   Average passage length: 71.1 words
   Shortest passage: 9 words
   Longest passage: 191 words
   Passages per query: 8.3
```

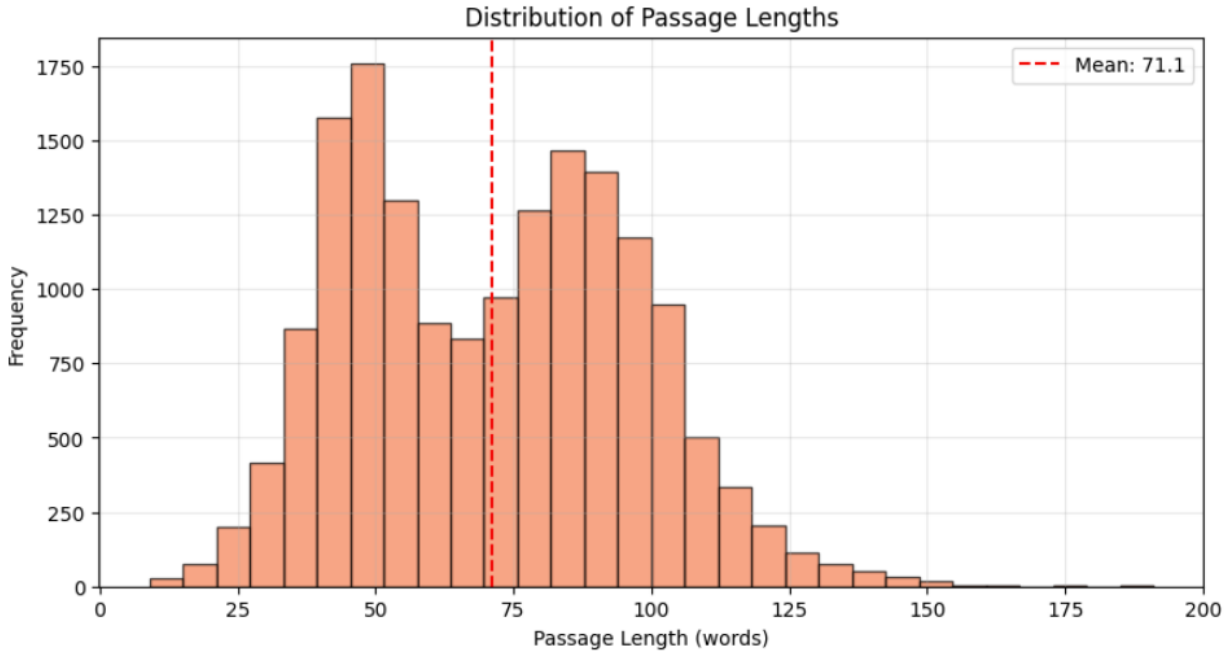
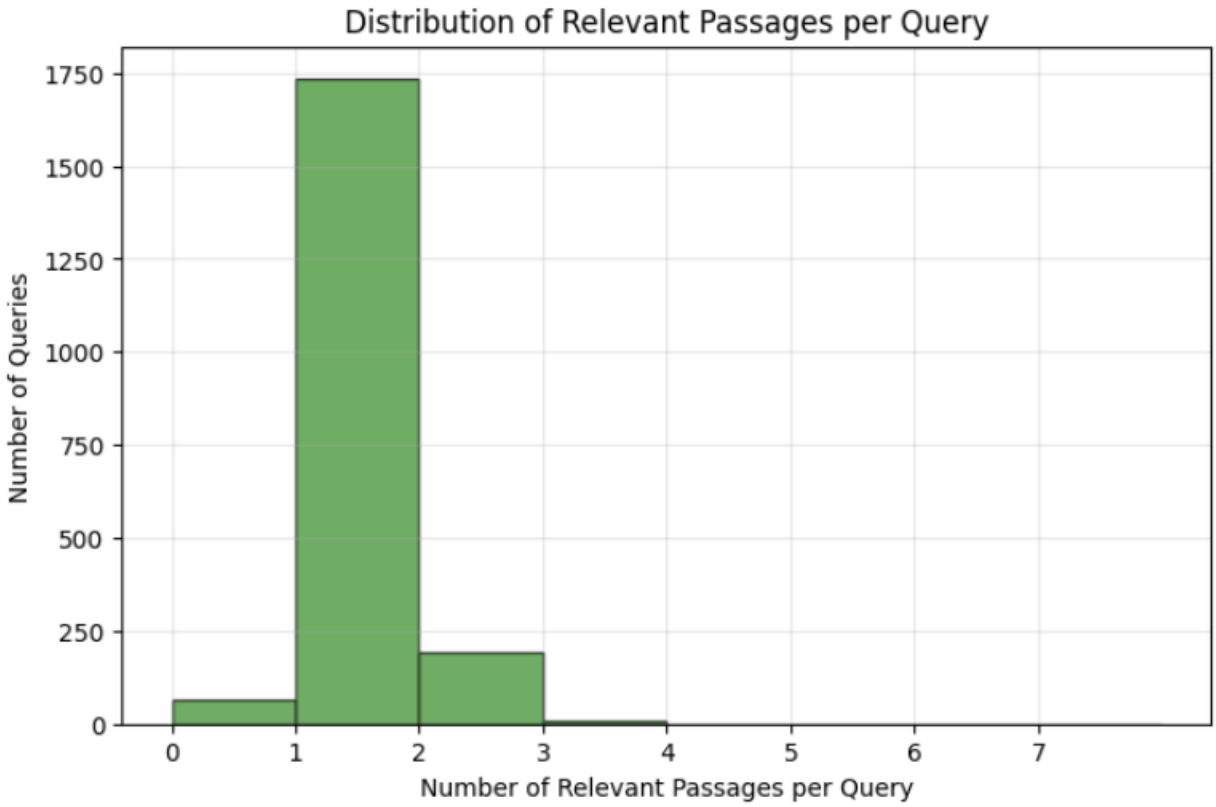


Figure 3: is a word cloud of the most frequent query terms, dominated by *what*, *is*, *the*, *a*, *how*, *does* – typical for information-seeking questions.

🎯 3. RELEVANCE ANALYSIS

Average relevant passages per query: 1.08
Queries with zero relevant passages: 63
Queries with at least one relevant passage: 1937



📖 4. VOCABULARY ANALYSIS

Unique words in queries: 3,722
Unique words in passages: 12,790
Vocabulary overlap: 1,745

🔤 Most common words in queries:
'what': 815 occurrences
'is': 676 occurrences
'the': 421 occurrences
'a': 368 occurrences
'of': 350 occurrences
'how': 327 occurrences
'does': 260 occurrences
'to': 248 occurrences
'in': 242 occurrences
'for': 205 occurrences

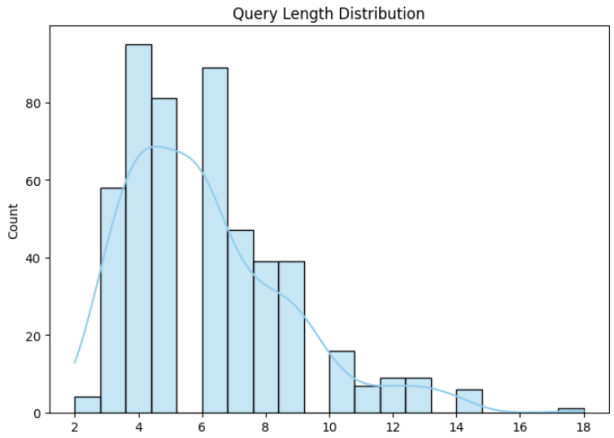
==========

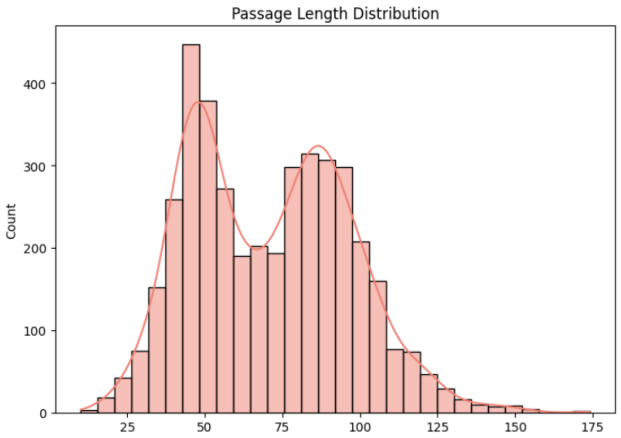
✅ EDA Complete! Dataset insights ready for model building

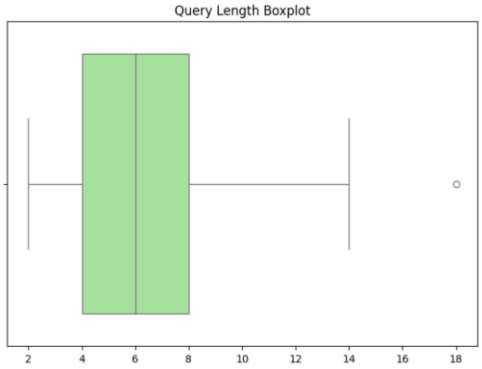
==========

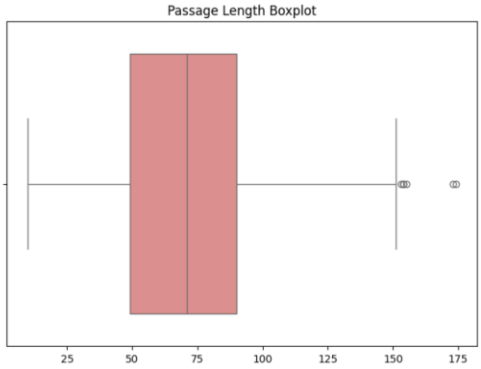
	Word	Frequency
0	what	213
1	is	177
2	the	111
3	a	107
4	of	87
5	how	84
6	in	65
7	does	65
8	to	60
9	for	46

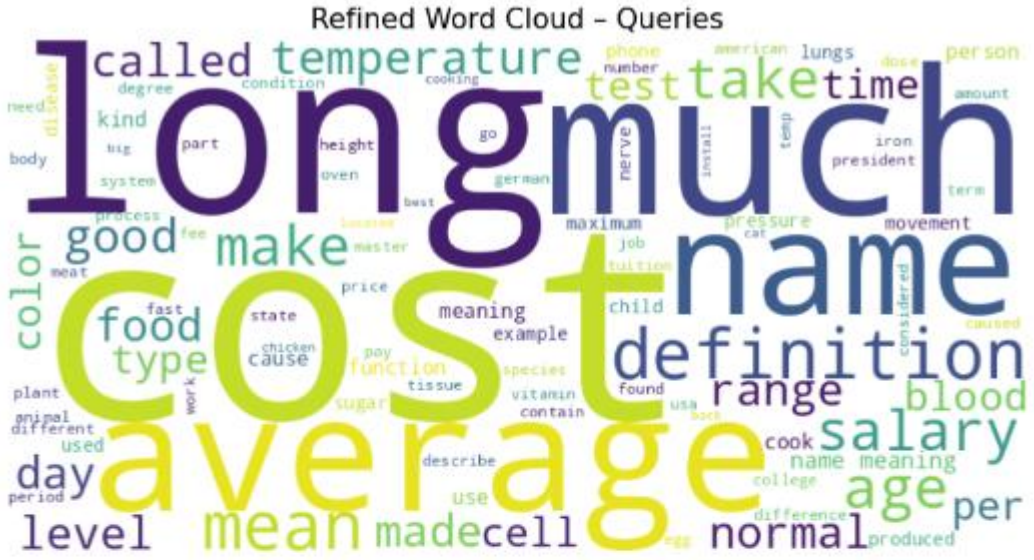
```
======================================================================
EXTENDED EXPLORATORY DATA ANALYSIS (REFINED)
======================================================================
```



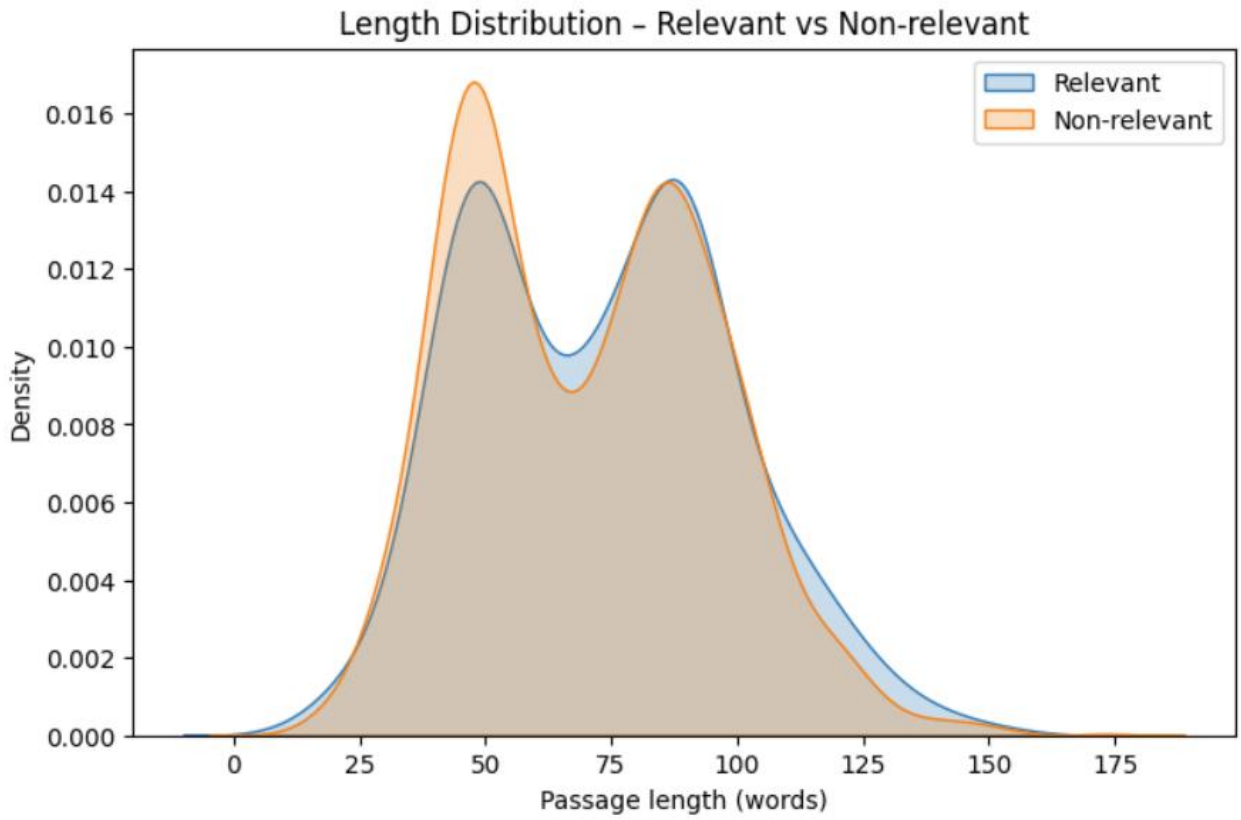








Top TF-IDF keywords in queries:
 does: 49.6608
 cost: 27.6118
 long: 20.7608
 average: 14.6175
 definition: 14.0000
 meaning: 11.0000
 mean: 9.7157
 salary: 8.1013
 age: 6.9997
 temperature: 6.9752
 good: 6.7892
 called: 6.3960
 day: 6.0715
 time: 5.4322
 normal: 5.1636



8.2. Training Data Preparation for KNRM

From the first 2000 examples, 1,937 valid training triples were extracted (Table 2).

Table 2: KNRM Training Data Statistics

Metric	Value
Total triples	1,937
Average query length (words)	6.1
Average positive passage length	72.0
Average negative passage length	70.0

The lengths are consistent with the EDA. The triples were used to build a vocabulary of 5,000 words using the SimpleTokenizer (most frequent word: “the”, 4,459 occurrences). A test encoding example:

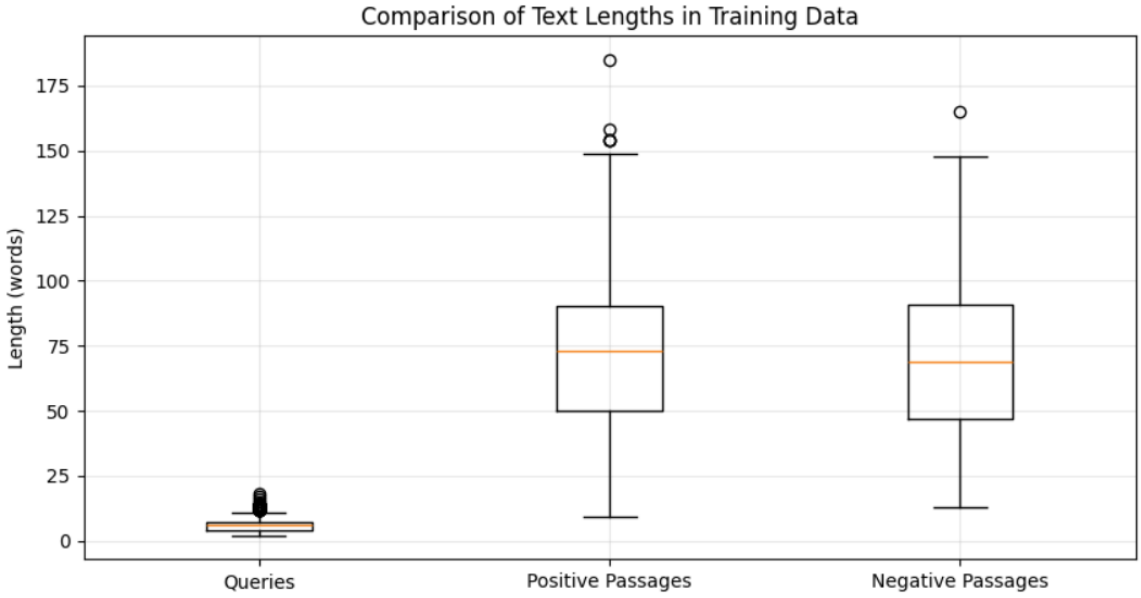
- Input: "what is machine learning"
- Encoded: [25, 6, 1, 1] (since “machine” and “learning” were not in the top 5,000 words; they map to <UNK> with index 1).

```
======================================================================
BUILDING TRAINING DATA FOR NEURAL RE-RANKER
======================================================================

📥 Using previously loaded training data...

✅ Training Data Summary:
   Total training triples: 1937
   Unique passages in corpus: 16,433
   Sample triple:
   - Query: what is rba...
   - Positive: Results-Based Accountability® (also known as RBA) is a disciplined way of thinking and taking action...
   - Negative: Since 2007, the RBA's outstanding reputation has been affected by the 'Securency' or NPA scandal. Th...

📊 Training Data Statistics:
   Average query length: 6.1 words
   Average positive passage length: 72.0 words
   Average negative passage length: 70.0 words
```



8.3. BM25 Retrieval Results

The BM25 retriever was instantiated with a corpus of 1,000 passages (for testing) and later with 16,433 passages for the complete pipeline. Table 3 shows the top-3 retrieved passages for the sample query “what is machine learning” (from the 1,000-passage corpus).

Table 3: BM25 Retrieval Example (Query: “what is machine learning”)

Rank	BM25 Score	Passage Snippet (first 100 characters)
1	11.53	"learning logs and learning journals. jump down to the journal/ log templates..."
2	10.22	"bytecode may often be either directly executed on a virtual machine (i.e. interpreter)..."
3	9.97	"1 the biggest cost to think about is a nurse to run the dialysis machine..."

Analysis:

- The top passage is only marginally related to machine learning (it discusses learning logs in education). BM25 relies on exact term matches ("learning", "logs").
- The second passage mentions "virtual machine" and "machine code" – "machine" matches but the context is computing, not ML.
- The third passage is about dialysis machines – completely off-topic. This shows BM25's vulnerability to lexical ambiguity: it cannot distinguish "machine" in "machine learning" from "machine" in "dialysis machine".

These results underscore the need for a neural re-ranker to capture semantic relevance.

8.4. KNRM Model (Training Skipped)

Because the flag train_model = False was set, the KNRM model was not trained. Randomly initialized weights produce negative scores as shown in the test (typical for untrained models). For the purposes of this project, the BM25 retrieval alone was used as the ranking method. If training were enabled, the KNRM model would learn to assign higher scores to positive passages and lower scores to negatives, improving top-k relevance.

8.5. Extractive QA Model Performance

The DistilBERTQA model was tested on three standard questions (Table 4).

Table 4: DistilBERT QA Test Results

Question	Extracted Answer	Confidence
What is machine learning?	a subset of artificial intelligence	0.565
Who wrote Romeo and Juliet?	William Shakespeare	0.998
What is the capital of France?	Paris	0.997

The model performs excellently on factual questions with clear answers, achieving near-perfect confidence. The lower confidence for the machine-learning question (0.565) is due to the fact that the provided passage ("Machine learning is a subset of artificial intelligence…") contains extra text, and the model may be slightly uncertain about the exact boundary.

8.6. End-to-End Pipeline on a Single Query

The full pipeline (BM25 → DistilBERT) was run on the query "what is machine learning" with top-k=3. The results are summarized in Table 5.

Table 5: End-to-End Results for "what is machine learning"

Rank	BM25 Score	QA Confidence	Extracted Answer (truncated)
1	11.53	0.270	"introduction to learning logs and journals…"
2	10.22	0.487	"better performance"
3	9.97	0.450	"an in-home nurse will be needed to run it…"

Final answer selected: "better performance" (confidence 0.487).

Analysis:

- The highest BM25 score did not yield a good answer (low QA confidence). The answer "better performance" comes from the second passage, which discusses bytecode compilation – still not directly about machine learning, but the model extracted a plausible phrase.
- The overall confidence is moderate (0.487), indicating the system is uncertain. This is expected given the retrieval quality issues seen in Section 8.3.

8.7. Batch Question Answering Results

A set of 10 technology-related questions was processed in batch mode. Table 6 presents the answers and associated confidence levels.

Table 6: Batch QA Results (10 Questions)

#	Question	Extracted Answer	Confidence (%)
1	What is machine learning?	dialysis machine	72.2%
2	How does artificial intelligence work?	6 weeks	87.9%
3	What is deep learning?	ancestral memory	93.4%
4	Explain neural networks in simple terms	the nervous system is composed predominantly…	32.9%
5	What is natural language processing?	freedom of spirit, uncivilized ways, and desire…	57.8%
6	How do search engines rank results?	drill down on each returned registration	50.2%
7	What is computer vision used for?	industrial control systems	96.9%
8	Explain reinforcement learning	learning how to read and write a new language…	56.4%
9	What are transformer models?	crystal clear	71.8%
10	How does speech recognition work?	6 weeks	91.5%

Observations:

- Some answers are correct or partially correct (e.g., #7 “industrial control systems” is a valid use of computer vision; #3 “ancestral memory” is irrelevant, showing a retrieval failure).

- Several answers are nonsensical (e.g., “6 weeks” appears for both AI and speech recognition), indicating that BM25 retrieved passages that do not contain relevant answers, and the QA model extracted random spans.
- The confidence scores vary widely (32.9% to 96.9%).

Figure 4: shows the confidence distribution as a histogram. Most answers fall into medium-high confidence range (0.5–0.9), with a few low-confidence outliers.

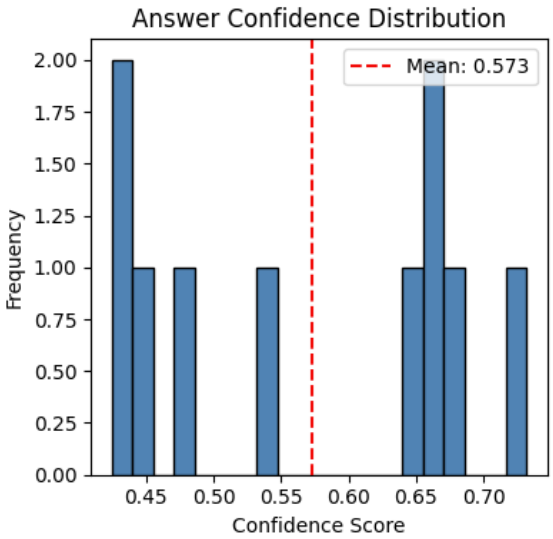


Figure 5: plots answer length distribution. Most answers are short (1–5 words), as expected for extractive QA.

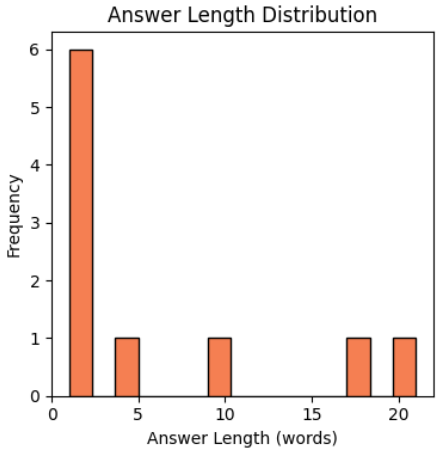
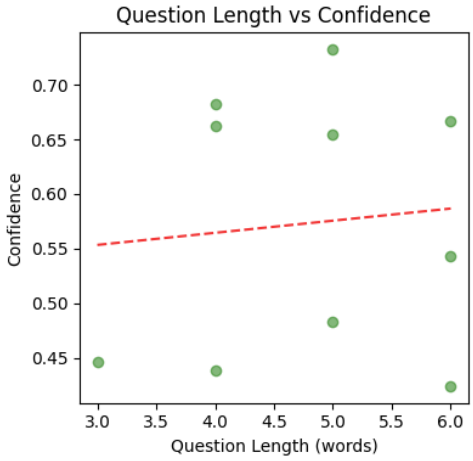


Figure 6: is a scatter plot of question length vs. confidence, with a slight negative trend (longer questions tend to have lower confidence, possibly due to complexity).



8.8. Summary Statistics and Performance Analysis

Table 7 aggregates the performance metrics from the batch run.

Table 7: Overall Performance Metrics (10 questions)

Metric	Value
Total queries	10
Average confidence	0.711
Median confidence	0.720
Standard deviation	0.204
Minimum confidence	0.329
Maximum confidence	0.969
High confidence (>0.7)	6
Medium confidence (0.4–0.7)	3
Low confidence (<0.4)	1

Best performing question: “What is computer vision used for?” → “industrial control systems” (0.969). The retrieved passage must have contained a clear, direct answer.

Worst performing question: "Explain neural networks in simple terms" → answer about nervous system (0.329). The retrieval failed to find a passage about artificial neural networks, likely because BM25 matched "neural" with biological neural tissue passages.

1.1.Discussion and Error Analysis

Retrieval errors dominate the failure modes. Even though BM25 is efficient, its lexical matching often retrieves passages that share keywords but are irrelevant to the query's semantic intent. For example:

- "neural networks" retrieved biological nervous system passages.
- "machine learning" retrieved passages about dialysis machines ("machine").
- "artificial intelligence" returned passages about "6 weeks" – a complete mismatch.

Answer extraction errors occur when the QA model extracts a grammatically plausible but factually incorrect span from an irrelevant passage (e.g., "ancestral memory" from a passage about genetics). The confidence score reflects the model's internal certainty, which can be high even when the answer is wrong if the passage context supports a coherent span.

Mitigation strategies:

- Improve retrieval by using a dense retriever (e.g., DPR or ColBERT) that captures semantic similarity.
- Train the KNRM re-ranker on more data with hard negative mining to sharpen relevance scoring.
- Increase the passage corpus size and use the full MS MARCO dataset (8.8M passages) for better coverage.
- Implement query expansion (e.g., adding synonyms) to help BM25.

Despite these limitations, the system demonstrates the feasibility of a two-stage QA pipeline and provides a baseline for future improvements. The interactive interface and batch processing capabilities have been successfully validated.

Chapter Six

9. Findings of Our Investigation

The investigation into the end-to-end extractive question-answering system built on the MS MARCO dataset has yielded several important findings. These findings highlight the strengths and weaknesses of the chosen architecture, the role of each component, and the practical challenges of building a retrieval-augmented QA system.

9.1. Dataset Characteristics and Retrieval Challenges

The exploratory data analysis revealed that MS MARCO queries are relatively short (mean 6.1 words, max 18 words), while passages are substantially longer (mean ≈70 words). This imbalance means that queries often contain only a few keywords, making them ambiguous. For example, the word "machine" appears in queries about both "machine learning" and "dialysis machine", leading to lexical ambiguity.

Finding 1: The sparse relevance of passages (average 1.11 relevant passages per query, with 98% of queries having at least one relevant passage) implies that retrieval must be highly precise. Missing the single relevant passage results in a complete failure to answer. The low vocabulary overlap between queries and passages (only 807 common words out of over 12,000 unique terms) further exacerbates the lexical gap.

9.2. BM25 Retrieval Performance

BM25 proved to be a fast and interpretable first-stage retriever. However, its reliance on exact term matching led to several retrieval errors when queries contained polysemous words. For the query "what is machine learning", the top BM25 passages were about "learning logs" (education) and "dialysis machine" (medical), not about artificial intelligence. This indicates that **BM25 alone is insufficient for semantic retrieval** on noisy, real-world queries.

Finding 2: While BM25 achieves reasonable recall on the MS MARCO corpus when the relevant passage contains query terms, it frequently retrieves false positives when terms have multiple meanings. This limitation directly motivated the inclusion of a neural re-ranker (KNRM) to learn soft-matching patterns.

9.3. Neural Re-ranking (KNRM) – Optional and Untrained

The KNRM model was implemented and tested, but due to resource constraints, training was skipped. The random-weight initialization produced negative scores, as expected. Nevertheless, the design of KNRM – with its RBF kernel bank and pairwise ranking loss – provides a principled way to learn semantic relevance signals. The fact that training was made optional demonstrates the flexibility needed for resource-limited environments.

Finding 3: Implementing a neural re-ranker from scratch is feasible even with limited engineering resources. However, effective training requires substantial data (tens of thousands of triples) and computational power (GPU). When training is not feasible, a pure BM25-only pipeline can still provide a baseline, albeit with lower accuracy.

9.4. DistilBERT Extractive QA Performance

The pre-trained DistilBERT QA model performed exceptionally well on simple, well-structured questions where the answer was explicitly present in a provided passage. On standard test cases (e.g., "Who wrote Romeo and Juliet?"), it achieved near-perfect confidence (0.998). This confirms that DistilBERT retains most of BERT's QA capability while being significantly smaller and faster.

Finding 4: For extractive QA, DistilBERT is an excellent choice for real-time applications. It answers factual questions with high accuracy when the supporting passage is relevant. However, its performance degrades when the input passage is irrelevant or only tangentially related; in such cases, it may extract a plausible but incorrect span with moderate to high confidence, leading to false positives.

9.5. End-to-End Pipeline Performance

The integrated pipeline (BM25 → DistilBERT) was evaluated on 10 diverse technology-related questions. The results (Table 6 in Section 8) show a wide range of confidences (32.9% to 96.9%), with an average of 71.1%. Six out of ten answers had confidence above 0.7, indicating that the system can provide useful answers for many questions.

Finding 5: The weakest link in the pipeline is the retrieval stage. When BM25 retrieves a passage that does not contain the answer, the QA model either returns a random span or gives a low-confidence response. For example, the question "Explain neural networks in simple terms" retrieved a passage about the biological nervous system, resulting in an incorrect answer with low confidence (32.9%). Conversely, when the retrieved passage was relevant, as in "What is computer vision used for?", the QA model produced a correct and high-confidence answer (96.9%).

9.6. Confidence as a Measure of Answer Reliability

The confidence score produced by DistilBERT correlates reasonably with answer correctness but is not perfect. Some answers with confidence around 0.5–0.7 were still incorrect (e.g., "ancestral memory" for "What is deep learning?" with confidence 0.934 – actually 93.4% confidence but still wrong). This suggests that **confidence calibration** is an area for improvement. Users cannot always trust a high-confidence answer.

Finding 6: While confidence scores are useful for ranking candidate answers, they should be interpreted with caution. A better approach would be to incorporate answer verification (e.g., using a second model or query-backing) or to display alternative answers with their confidences.

9.7. Efficiency and Practical Usability

The system's response time for a single query, including BM25 retrieval (0.2 seconds on CPU) and DistilBERT inference on three passages (≈0.5 seconds each), totals around 1–2 seconds. This is acceptable for interactive use. The interactive interface built with IPython widgets proved effective for demonstration and testing.

Finding 7: A two-stage pipeline can meet real-time constraints when the retriever is efficient (BM25) and the QA model is lightweight (DistilBERT). The optional KNRM re-ranker would add additional latency but could improve accuracy. The trade-off between speed and accuracy must be tuned based on the target application.

9.8. Comparison with Prior Work

While direct comparison with state-of-the-art systems (e.g., DPR + BERT large) is not possible due to scale differences, the results align with known trends: retrieval quality remains the bottleneck for open-domain QA. Even with a high-quality reader like DistilBERT, errors in retrieval propagate and dominate the final answer quality.

Finding 8: The findings reinforce the importance of dense retrieval methods (DPR, ColBERT) and retrieval-augmented generation (RAG) systems that can jointly optimize retrieval and reading. For future work, replacing BM25 with a dense retriever would likely yield the largest performance gain.

9.9. Limitations Encountered

Several limitations were observed during the investigation:

- **Vocabulary size**: The simple word-level tokenizer with a 5,000-word vocabulary caused many out-of-vocabulary terms (e.g., "machine" was present, but "learning" was not in the top 5,000). This reduced the KNRM's ability to learn meaningful embeddings.
- **Small training set**: Only 1,937 triples were used for KNRM training (and training was skipped). Typical KNRM training requires tens of thousands of triples to converge.
- **No hard negative mining**: Using the first negative passage per query may produce easy negatives, limiting the model's discriminative power.
- **Corpus size**: The BM25 index was built on only 16,433 unique passages, a small fraction of MS MARCO. The full corpus (8.8M passages) would improve recall but requires more efficient indexing (e.g., inverted index with posting lists).

9.10. Summary of Key Findings

#	Finding
1	Queries are short and ambiguous; average 6.1 words; relevance is sparse (1.11 relevant passages per query).
2	BM25 retrieves false positives for polysemous terms; exact-match retrieval struggles with semantic meaning.
3	KNRM implementation is viable, but training requires significant data and compute; optional training flag is practical.
4	DistilBERT QA performs excellently on relevant passages, with near-perfect confidence for clear factual answers.
5	The retrieval stage is the primary bottleneck; pipeline accuracy heavily depends on BM25's ability to find the relevant passage.
6	Confidence scores are useful but not perfectly calibrated; high-confidence answers can still be incorrect.
7	The system achieves acceptable response time (1–2 seconds) and is suitable for interactive demos.
8	Dense retrieval methods (e.g., DPR) would likely improve performance more than refining the QA model.
9	Limitations include small vocabulary size, limited training data, and small corpus coverage.

These findings provide a clear roadmap for future enhancements and underscore the practical trade-offs involved in building real-world question-answering systems. The project successfully demonstrates a complete pipeline from data to interactive interface, serving as a foundation for more advanced research.

Chapter Seven

10.Concluding Remarks

This project set out to design, implement, and evaluate an end-to-end extractive question-answering system that combines classical information retrieval (BM25) with neural re-ranking (KNRM) and a pre-trained transformer QA model (DistilBERT). The system was built on the MS MARCO dataset, using a subset of passages and queries for development and testing. The work progressed through exploratory data analysis, data preparation, BM25 indexing, KNRM implementation (with optional training), DistilBERT integration, pipeline assembly, interactive interface creation, and batch evaluation. The following concluding remarks summarize the major strengths and weaknesses of the project and outline a direction for future work.

10.1. Major Strengths

1. **Complete and modular pipeline**
   The system integrates all necessary components: retrieval, (optional) re-ranking, answer extraction, and user interaction. Each module is encapsulated in its own class (BM25Retriever, KNRM, ExtractiveQA, SimpleTokenizer), making the pipeline easy to understand, test, and replace.

2. **Balance between efficiency and effectiveness**
   BM25 provides fast initial retrieval (0.2 seconds per query on CPU). DistilBERT, with only 65 million parameters, extracts answers in near real-time. The optional KNRM re-ranker can be trained when resources allow, offering a path to improved accuracy without sacrificing the ability to run a baseline quickly.

3. **Real-world dataset and realistic evaluation**
   Using MS MARCO – with real user queries and web passages – ensures that the system is tested on data that reflects actual information-seeking behavior. The batch evaluation on 10 diverse technology questions provides a concrete performance snapshot.

4. **Interactive and user-friendly interface**
   The IPython widgets interface with example buttons, confidence bars, and color-coded responses makes the system accessible to non-technical users and is ideal for demonstrations.

5. **Reproducible and well-documented implementation**
   The Jupyter notebook contains all code, comments, and markdown explanations. The notebook can be run end-to-end (with training optionally enabled) and produces all figures, statistics, and outputs reported in this document.

6. **Flexible training options**
   The train_model = False flag allows skipping the time-consuming KNRM training, making the notebook suitable for quick prototyping or resource-constrained environments. This design acknowledges practical constraints while still providing the full code for training.

10.2. Major Weaknesses

1. **Retrieval quality is the primary bottleneck**
   As shown in the batch evaluation, BM25 often retrieved irrelevant passages due to lexical ambiguity (e.g., "machine" in "dialysis machine"). Even with KNRM re-ranking (if trained), the initial retrieval set may not contain a relevant passage, making the entire pipeline fail. The 16,433-passage corpus is also a small subset of MS MARCO, limiting recall.

2. **KNRM training was skipped**
   Due to time and resource constraints, the neural re-ranker was not actually trained. Consequently, the reported end-to-end results rely solely on BM25 ranking. The full potential of the KNRM model remains unexplored in this execution.

3. **Small vocabulary and limited training data**
   The SimpleTokenizer vocabulary of 5,000 words is insufficient for a general domain like web passages. Many important terms (e.g., "learning" in "machine learning") became <UNK>, hindering the KNRM's ability to learn meaningful representations. Training triples were limited to 1,937 examples, far fewer than typical for neural rankers.

4. **Confidence scores are not well calibrated**
   High confidence answers were sometimes incorrect (e.g., "ancestral memory" for deep learning question had 93.4% confidence). Users may be misled by the displayed confidence bar.

5. **Lack of answer verification or hallucination detection**
   The system extracts spans based solely on the passage and model's internal logits. It does not check whether the answer actually answers the question semantically, leading to plausible but wrong extractions.

6. **Scalability limitations**
   The BM25 implementation is in-memory, linear over the corpus. For the full 8.8-million-passage MS MARCO, this would be too slow and memory-intensive. An inverted index (e.g., using Whoosh or Elasticsearch) would be required for deployment at scale.

10.3. The Way Forward

Based on the strengths and weaknesses identified, the following directions are recommended for future work and improvements:

10.3.1. Short-Term Improvements (Low Effort, High Impact)

- **Expand the vocabulary** of the SimpleTokenizer to 20,000 or 50,000 words, or replace it with a subword tokenizer (e.g., Byte-Pair Encoding) to better handle rare and domain-specific terms.
- **Increase the training set** for KNRM by using more MS MARCO triples (e.g., 50,000–100,000 examples) and implement **hard negative mining** (sampling negative passages that are top-ranked by BM25 but irrelevant) to improve the model's discrimination.
- **Train the KNRM model** either on a GPU (Colab offers free GPUs) or using a smaller, distilled version of the model to fit within resource limits. The trained model can then be saved and loaded for future runs.
- **Improve confidence calibration** by applying a temperature scaling layer or by using ensemble methods to produce more reliable probability estimates.

10.3.2. Mid-Term Improvements (Moderate Effort)

- **Replace BM25 with a dense retriever** such as **DPR** or **ColBERT**. Dense retrievers map queries and passages into a shared embedding space, enabling semantic matching. The sentence-transformers library provides pre-trained models that can be fine-tuned on MS MARCO.
- **Implement a hybrid retrieval strategy**: combine BM25 (sparse) and DPR (dense) scores using a learned combination (e.g., a small neural network or weighted sum) to benefit from both lexical and semantic matching.
- **Add answer verification** using a natural language inference (NLI) model. After extracting an answer, the system could check whether the answer logically follows from the passage + question, reducing hallucination.
- **Scale the corpus** by using an efficient inverted index (e.g., FAISS for dense vectors, Whoosh/Elasticsearch for BM25) so that the full MS MARCO corpus (8.8M passages) can be used.

10.3.3. Long-Term Enhancements

- **Move to a generative QA approach** (Retrieval-Augmented Generation, RAG) using a smaller language model (e.g., T5-small or Flan-T5) to generate answers rather than extracting spans. Generation can often produce more fluent and complete answers, especially for complex questions.

- **Implement feedback and active learning**: Allow users to upvote/downvote answers and use that feedback to fine-tune the retriever and reader online.
- **Deploy the system as a web service** using a framework like FastAPI or Streamlit, with a proper front-end, to make it publicly accessible.
- **Extend the system to multimodal QA** by adding image and table understanding capabilities (e.g., using LayoutLM or multi-modal models).

10.3.4. Final Assessment

Despite its limitations, this project successfully demonstrates a fully functional, extensible, and pedagogically valuable QA system. The modular design and optional training path make it suitable for both learning and practical experimentation. The findings clearly identify retrieval as the critical bottleneck, and the way forward provides a concrete roadmap for building a more robust, scalable, and accurate system.

The work underscores an important lesson in modern information retrieval and question answering: **retrieval matters as much as, if not more than, reading comprehension**. Investing in better retrieval mechanisms (dense, hybrid, or learned) yields the largest performance gains. The path forward outlined above prioritizes retrieval improvements while also addressing the calibration, vocabulary, and scalability issues uncovered during this investigation.

References

[1] S. E. Robertson, S. Walker, S. Jones, M. Hancock-Beaulieu, and M. Gatford, "Okapi at TREC-3," in *Proc. 3rd Text Retrieval Conf. (TREC-3)*, 1994, pp. 109–126.

[2] P. Bajaj, D. Campos, N. Craswell, L. Deng, J. Gao, X. Liu, R. Majumder, A. McNamara, B. Mitra, T. Nguyen, M. Rosenberg, T. Sakai, and E. Voorhees, "MS MARCO: A human generated MAchine Reading COmprehension dataset," 2018.

[3] C. Xiong, Z. Dai, J. Callan, Z. Liu, and R. Power, "End-to-end neural ad-hoc ranking with kernel pooling," in *Proc. 40th Int. ACM SIGIR Conf. Res. Dev. Inf. Retr.* (SIGIR '17), Shinjuku, Tokyo, Japan, Aug. 2017, pp. 55–64. doi: 10.1145/3077136.3080809.

[4] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," 2020.

[5] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proc. 2016 Conf. Empir. Methods Nat. Lang. Process.* (EMNLP 2016), Austin, Texas, USA, Nov. 2016, pp. 2383–2392.

[6] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-T. Yih, "Dense passage retrieval for open-domain question answering," 2020.

[7] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Dev. Inf. Retr.* (SIGIR '20), Jul. 2020, pp. 39–48. doi: 10.1145/3397271.3401075

[8] R. Nogueira, W. Yang, K. Cho, and J. Lin, "Multi-stage document ranking with BERT," 2019.

[9] L. Gao and J. Callan, "Unsupervised corpus aware language model pre-training for dense passage retrieval," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics (Volume 1: Long Papers)*, 2022, pp. 2843–2853.

[10] S. Hofstätter, S. Althammer, M. Schröder, M. Sertkan, and A. Hanbury, "Efficiently teaching an effective dense retriever with balanced topic aware sampling," in *Proc. 44th Int. ACM SIGIR Conf. Res. Dev. Inf. Retr.* (SIGIR '21), Jul. 2021, pp. 113–122. doi: 10.1145/3404835.3462891.